SOFTWARE SECURITY LABORATORY
HANYANG UNIVERSITY

String Security

2025년 2학기

한양대학교 컴퓨터소프트웨어학부
임을규

Strings

Character Strings

- the standard C library supports
  - strings of type `char`
  - wide strings of type `wchar_t`

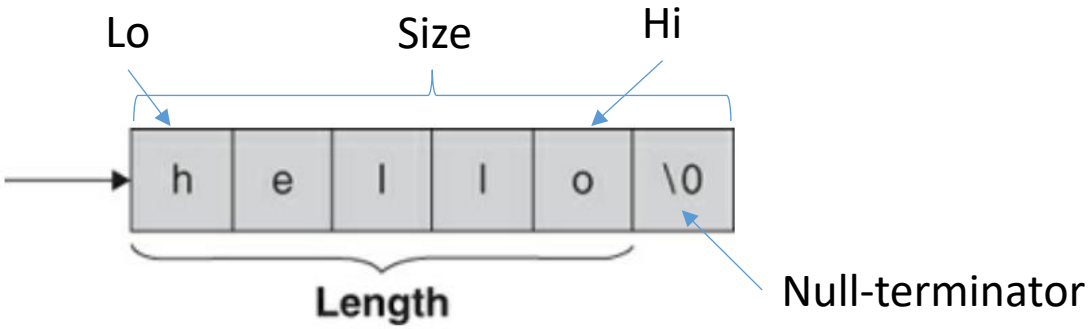


Figure 2.1. String representation of "hello"

An Example

- size of an array

```
void clear(int array[]) {
  for (size_t i = 0; i < sizeof(array) / sizeof(array[0]); ++i) {
     array[i] = 0;
   }
}

void dowork(void) {
  int dis[12];

  clear(dis);
  /* ... */
}
```

multibyte character set

- basic execution character set
  - 26 uppercase letters, 26 lowercase letters, …
- UTF-8
  - A multibyte character set
  - Backward compatible
  - 1 ~ 4 bytes
- String literals
  - L”xyz”
  - Type `wchar_t`

Strings

- `const char s[3] = “abc”;`
  - The size of array is 3
  - The trailing null byte is omitted
  - `const char s[ ] = “abc”;`

Strings in C++

- std::basic_string
  - a sequence of characters
  - `string` is
    - a `typedef` for the template specialization `basic_string<char>`
  - `wstring` is
    - a `typedef` for the template specialization `basic_string<wchar_t>`
- C++ string class
  - all common implementations are null-terminated
  - some other string types, such as Win32 `LSA_UNICODE_STRING`, do not have to be null-terminated

Character Types

- three types
  - `char, signed char, unsigned char`
- philosophy

`signed char` and `unsigned char`

- Suitable for small integer values

plain `char`

- The type of each element of a string literal
- Used for character data (where signedness has little meaning) as opposed to integer data

- Example

```
size_t len;
char cstr[] = "char string";
signed char scstr[] = "signed char string";
unsigned char ucstr[] = "unsigned char string";

len = strlen(cstr);
len = strlen(scstr);  /* warns when char is unsigned */
len = strlen(ucstr);  /* warns when char is signed */
```

int

- `EOF` (a negative value)
- or char data interpreted as `unsigned char`
- functions that return `int`
  - fgetc(), getc(), getchar(), fgetwc(), getwc(), getwchar()
- char classification functions declared in `<ctype.h>`
  - accept `int`
  - e.g. `isalpha()`

Example

- example

```
int main()
{
        char c = 'a';
        char b[] = “a”;
        wchar_t wc;
        printf(" size of 'a': %d\n", sizeof('a'));
        printf(" size of c: %d\n", sizeof(c));
        printf(" size of wc: %d\n", sizeof(wc));
}
```

- in C
  - sizeof(‘a’) → ???, sizeof(c) → ???
- in C++ ???

unsigned char

- useful when the object might be of any type
- `fwrite()`
- guaranteed to be represented using a pure binary notation

unsigned char

- Example

```
int main()
{   unsigned char buf1[6] = { 0xff, 0x56, 0x78, 0xfa, 0xf1, 0};
    unsigned char buf2[6];
    FILE *stream;
    stream = fopen(“out.dat”, “wb”);
    fwrite(buf1, sizeof(char), 6, stream);
    fclose(stream);
    stream = fopen(“out.dat”, “rb”);
    fread(buf2, sizeof(char), 6, stream); fclose(stream);
    for (int i=0; i<6 && buf2[i] != NULL; i++)
        printf(“%x “, buf2[i]);
    printf(“\n”);
    return 0;
 }
```

Sizing Strings

- essential in preventing buffer overflows
- The CERT C Secure Coding Standard
  - STR31-C. Guarantee that storage for strings has sufficient space for character data and the null terminator
- Terms
  - Size: number of bytes allocated to the array (same as `sizeof(array)`)
  - Count: number of elements in the array (same as the Visual Studio 2010 `_countof(array)`)
  - Length: number of characters before null terminator
    - strlen()

Example

```
wchar_t wide_str1[] = L”0123456789”;
wchar_t *wide_str2 = (wchar_t *) malloc (strlen(wide_str1) +1);
if (wide_str2 == NULL) {
    /* handle error */
}
strcpy(wide_str2, wide_str1);
/* … */
free(wide_str2);
wide_str2 = NULL;
```

Common String Manipulation Errors

- 4 common errors
  - **unbounded string copies**
  - **off-by-one errors**
  - null-termination errors
  - string truncation

Improperly Bounded String Copies

- when data is copied from a source to a fixed-length character array
  - ex) when reading from standard input into a fixed-length buffer
- `gets()` deprecated in C99 and eliminated from C11
- "STR35-C. Do not copy data from an unbounded source to a fixed-length array."
- Example 2.1. Reading from `stdin()`

```
#include <stdio.h>
#include <stdlib.h>

void get_y_or_n(void) {
  char response[8];
  puts("Continue? [y] n: ");
  gets(response);
  if (response[0] == 'n')
    exit(0);
  return;
}
```

Copying and concatenating strings

- standard lib functions
  - `strcpy(), strcat(), sprintf()`
- int main(int argc, char *argv[ ]) …
  - `argv[0] ~ argv[argc-1]` : pointers to null-terminated strings
  - Do not assume that any element of the `argv` array, including `argv[0]`, is not null.

```
int main(int argc, char *argv[]) {
  /* Do not assume that argv[0] can-
not be null */
  const char * const name =
argv[0] ? argv[0] : "";
  char *prog_name = (char *)mal-
loc(strlen(name) + 1);
  if (prog_name != NULL) {
    strcpy(prog_name, name);
  }
  else {
      /* Failed to allocate memory -
recover */
  }
  /* ... */
}
```

sprintf() function

- writes output to an array
- a null character is written at the end of the characters written
- it is often difficult to determine the max size required for the target array
  - ex) INT_MAX = 2,147,483,647 → at least 11 characters
  - negative integer
  - floating-point values are even more difficult to predict
- `snprintf()` function
  - additional parameter: `size_t n`
  - n-1 characters will be written because the null char will be added
  - if successful, returns the number of chars written (excluding the null char)
  - relatively secure, but susceptible to format string vulnerabilities

std::cin in C++

- also has problems

```
1  #include <iostream>
2
3  int main(void) {
4    char buf[12];
5
6    std::cin >> buf;
7    std::cout << "echo: " << buf << '\n';
8  }
```

- the field width can be set with `ios_base::width` or `setw()`
  - a null character is automatically appended after `n-1` characters are extracted

```
1  #include <iostream>
2
3  int main(void) {
4    char buf[12];
5
6    std::cin.width(12);
7    std::cin >> buf;
8    std::cout << "echo: " << buf << '\n';
9  }
```

Off-by-One Errors

- another common problem
- similar to unbounded string copies
- example

```
01  #include <string.h>
02  #include <stdio.h>
03  #include <stdlib.h>
04
05  int main(void) {
06    char s1[] = "012345678";
07    char s2[] = "0123456789";
08    char *dest;
09    int i;
10
11    strcpy_s(s1, sizeof(s2), s2);
12    dest = (char *)malloc(strlen(s1));
13    for (i=1; i <= 11; i++) {
14      dest[i] = s1[i];
15    }
16    dest[i] = '\0';
17    printf("dest = %s", dest);
18    /* ... */;
19  }
```

Null-Termination Errors

- Example

```
int main(void) {
  char a[16];
  char b[16];
  char c[16];
  strncpy(a, "0123456789abcdef", sizeof(a));
  strncpy(b, "0123456789abcdef", sizeof(b));
  strcpy(c, a);
  /* ... */
}
```

- `strncpy()` copies the first `n` characters
  - do not guarantee null-termination

String Vulnerabilities and Exploits

- String manipulation in C or C++
  - become dangerous when code operates on untrusted data from external sources
  - it is safer to view all external data as untrusted
- Example

```
bool IsPasswordOK(void) {
  char Password[12];

  gets(Password);
 r eturn 0 == strcmp(Password, "goodpass");
}

int main(void) {
  bool PwStatus;

  puts("Enter password:");
  PwStatus = IsPasswordOK();
  if (PwStatus == false) {
    puts("Access denied");
    exit(-1);
  }
}
```

- Example

Figure 2.2. Correct password grants access to user.

Figure 2.3. Incorrect password denies access to user.

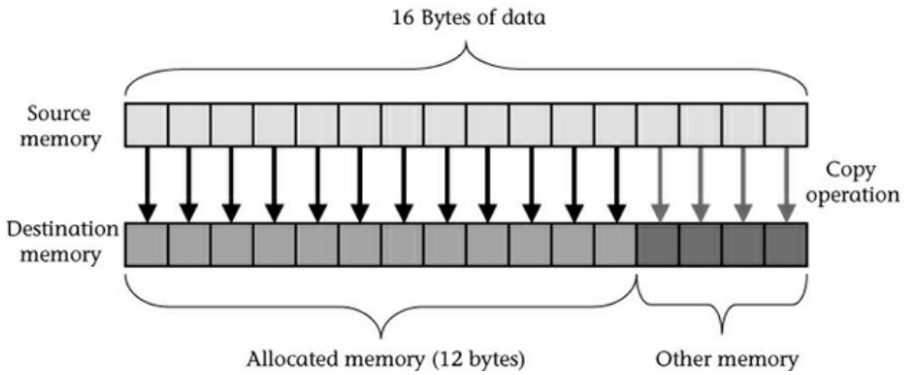


Figure 2.4. Copying 16 bytes of data into a 12-byte array

Buffer Overflows

- C and C++ are susceptible to buffer overflows because
  - define strings as null-terminated arrays of characters
  - do not perform implicit bounds checking
  - provide standard library calls for strings that do not enforce bounds checking

Process Memory Organization

can be marked **read-only**

initialized data, uninitialized data, static variables, global variables

dynamically allocating memory

to support process execution

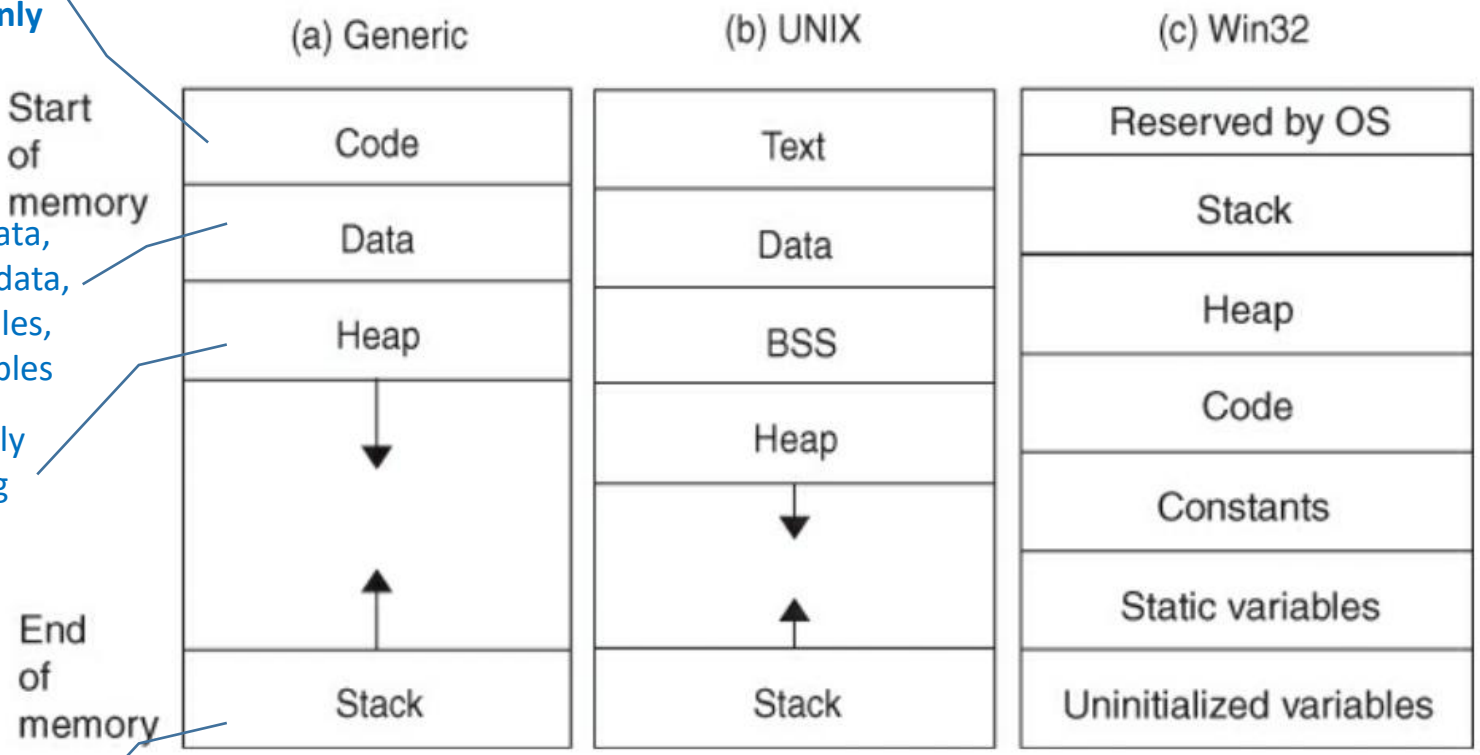


Figure 2.5. Process memory organization

Programs and Processes

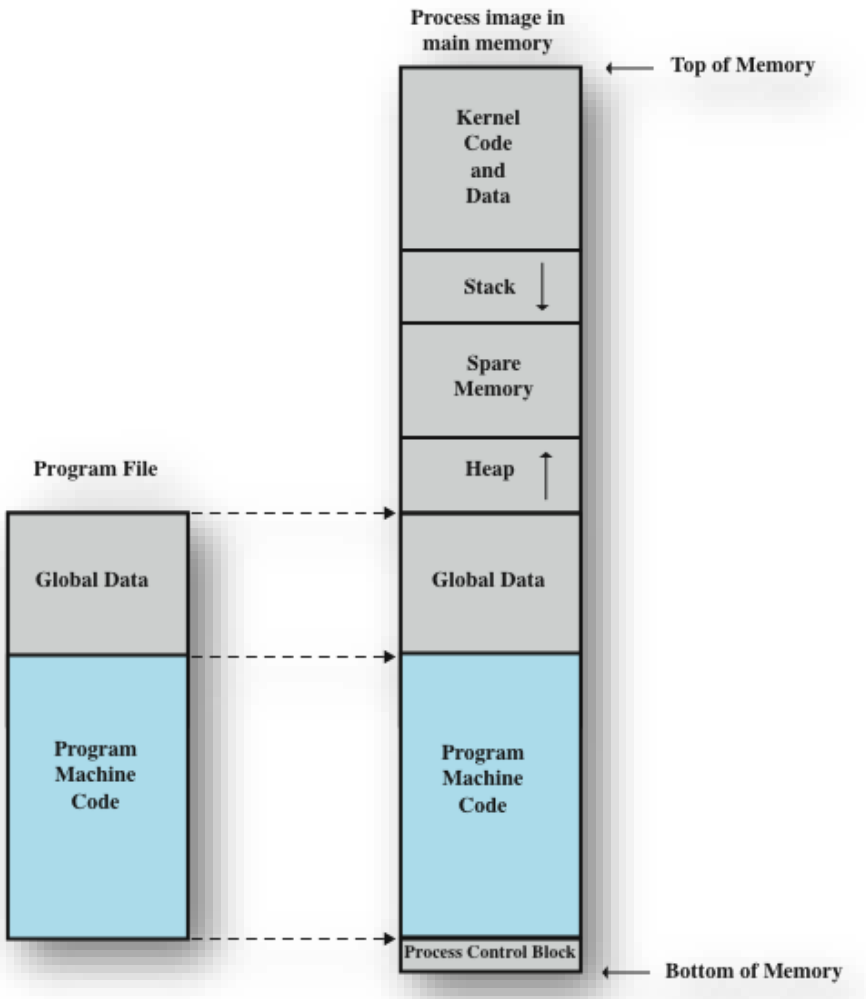


Figure 10.4 Program Loading into Process Memory

Stack Buffer Overflows

- occur when buffer is located on stack
  - also referred to as *stack smashing*
  - used by Morris Worm
  - exploits included an unchecked buffer overflow
- are still being widely exploited
- stack frame
  - when one function calls another it needs somewhere to save the return address
  - also needs locations to save the parameters to be passed in to the called function and to possibly save register values

Stack Frame with Functions P and Q

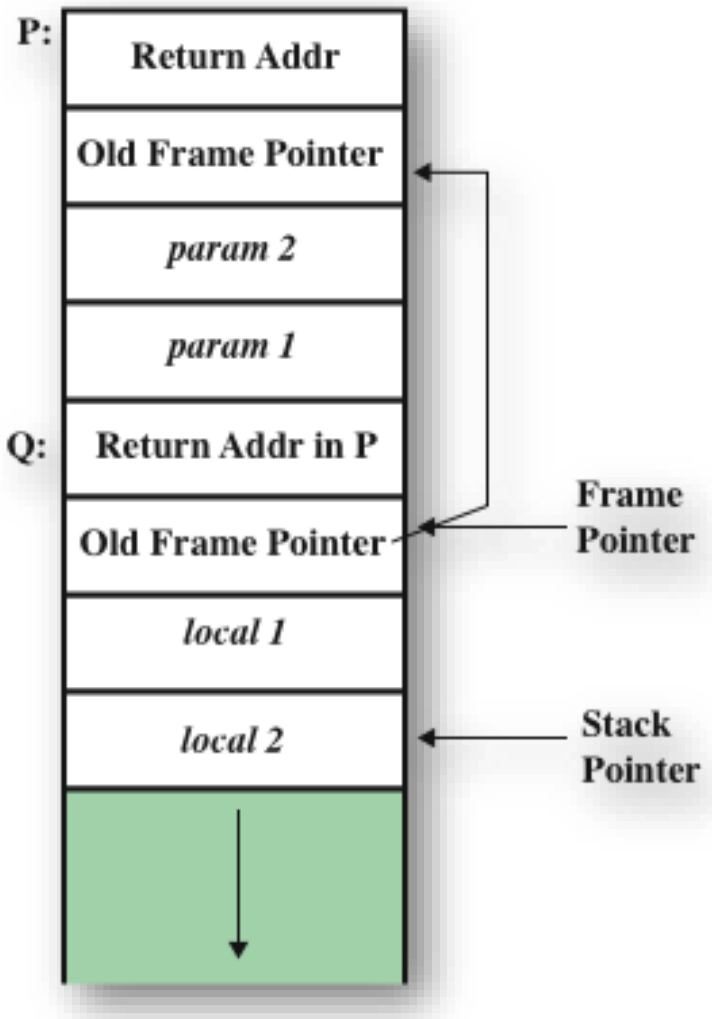


Figure 10.3 Example Stack Frame with Functions P and Q

Stack Management

- stack supports program execution
  - by maintaining automatic process-state data

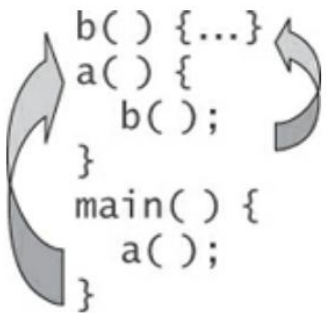


Figure 2.6. Stack management

  - information in the stack frame
    - return address
    - local variables
    - frame pointer

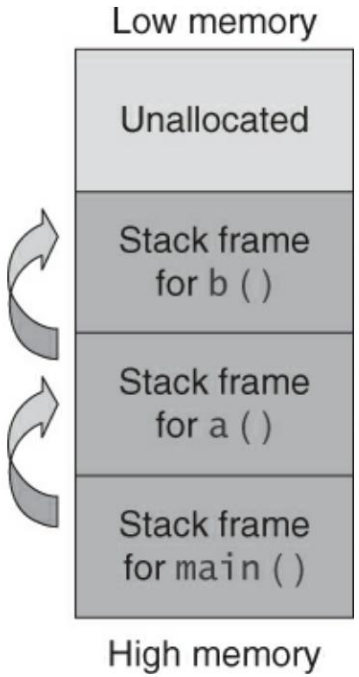


Figure 2.7. Calling a subroutine

Stack Management

Disassembly using Intel Notation

```
void foo(int, char *); // function prototype

int main(void) {
  int MyInt=1; // stack variable located at ebp-8
  char *MyStrPtr="MyString"; // stack var at ebp-4
  /* ... */
  foo(MyInt, MyStrPtr); // call foo function
    mov  eax, [ebp-4]
    push eax            # Push 2nd argument on stack
    mov  ecx, [ebp-8]
    push ecx            # Push 1st argument on stack
    call foo            # Push the return address on stack and
                        # jump to that address
    add  esp, 8
  /* ... */
}
```

MyStrPtr variable

MyInt variable

After returning from foo()

Stack Management

- foo() function prologue

```
void foo(int i, char *name) {
  char LocalChar[24];
  int LocalInt;
    push ebp        # Save the frame pointer.
    mov ebp, esp    # Frame pointer for subroutine is set to the
                    # current stack pointer.
    sub esp, 28     # Allocates space for local variables.
  /* ... */
```

- foo() function Epilogue

```
/* ... */
 return;
   mov  esp, ebp    # Restores the stack pointer.
   pop  ebp         # Restores the frame pointer.
   ret              # Pops the return address off the stack
                    # and transfers control to that location.
}
```

Stack Management

- Stack frame after invoking foo()

Table 2.2. Stack Frame for foo() following Execution of the Function Prologue

Address	Value	Description	Length
0x0012FF4C	?	Last local variable—integer: LocalInt	4
0x0012FF50	?	First local variable—string: LocalChar	24
0x0012FF68	0x12FF80	Calling frame of calling function: main()	4
0x0012FF6C	0x401040	Return address of calling function: main()	4
0x0012FF70	1	First argument: MyInt (int)	4
0x0012FF74	0x40703C	Second argument: pointer toMyString (char *)	4

Stack Smashing

- **Stack smashing** occurs when a buffer overflow overwrites data in the memory allocated to the execution stack
- a buffer overflow in the stack segment can lead to an attacker executing arbitrary code
- Example

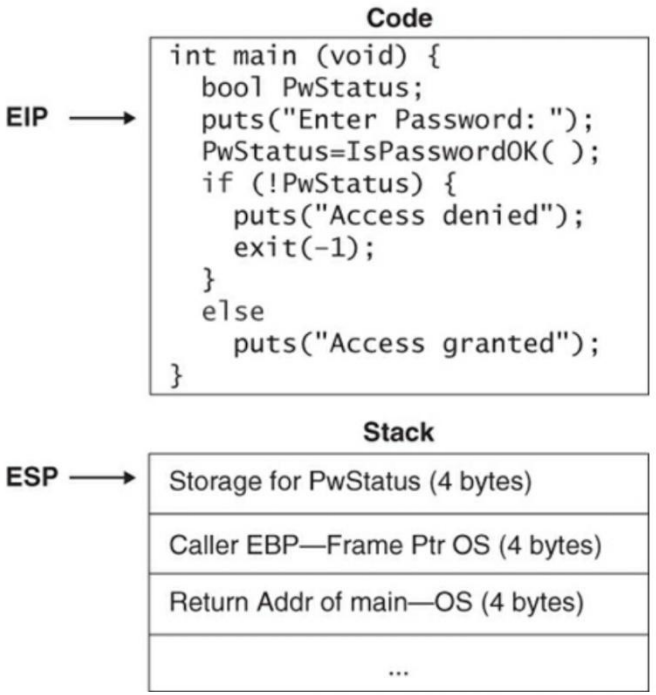


Figure 2.8. The stack before `IsPassWordOK()` is called

Stack Smashing

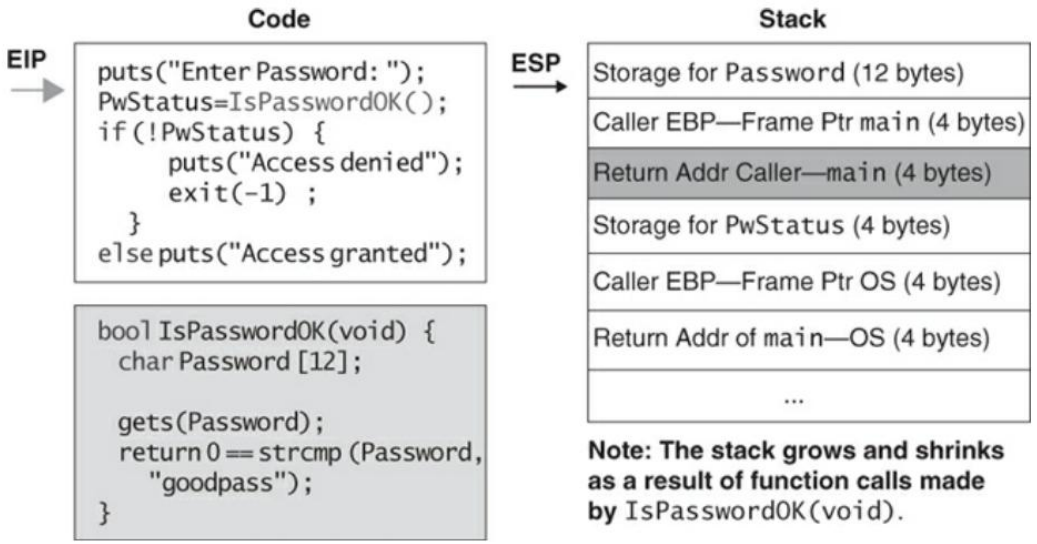
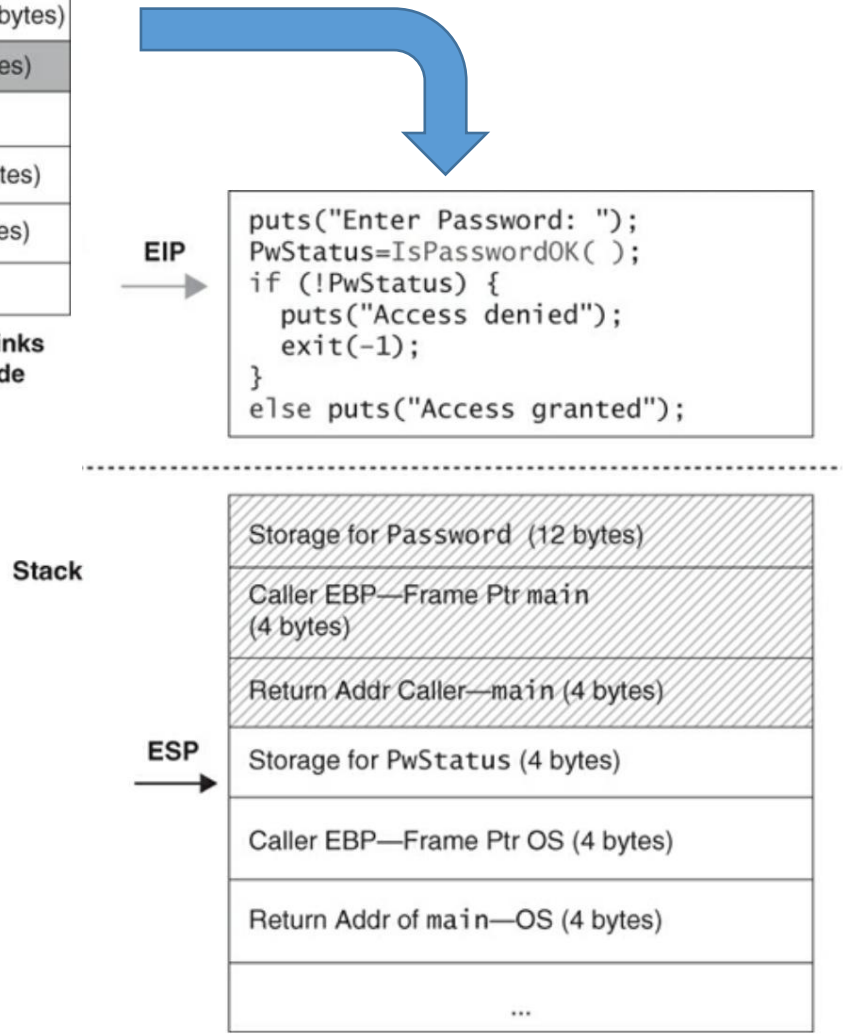


Figure 2.9. Information in stack while `IsPasswordOK()` is executed



Corrupted program stack

Figure 2.11. An improperly bounded `Password` array crashes the program if its character limit is exceeded.

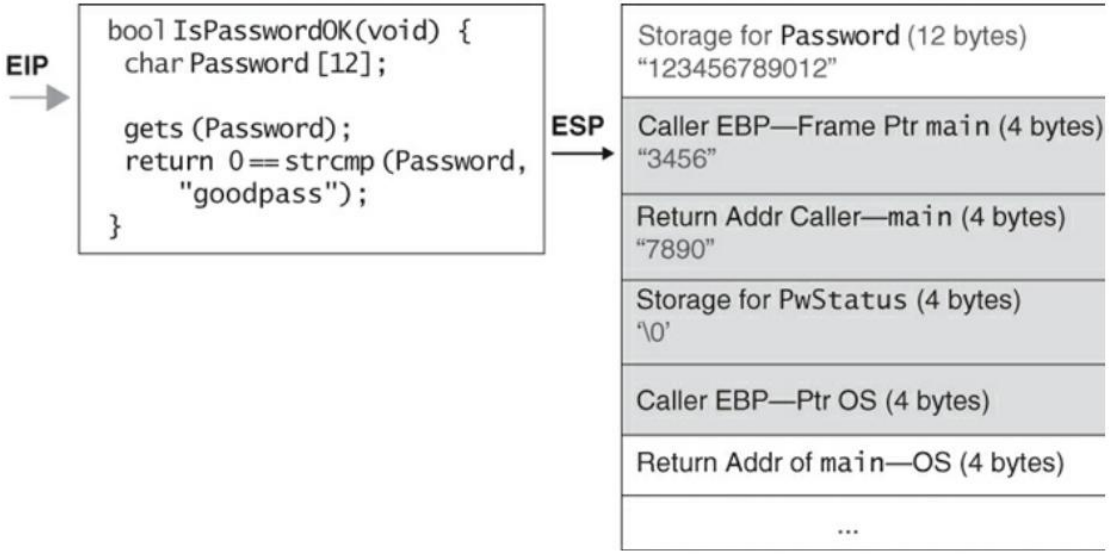


Figure 2.12. Corrupted program stack

Corrupted program stack


```
C:\WINDOWS\system32\cmd.exe
C:\BufferOverflow\Release>BufferOverflow.exe
Enter Password:
12345678901234 56j▶*!
Access granted

C:\BufferOverflow\Release>
```

Line	Statement
1	puts("Enter Password: ");
2	PwStatus=IsPasswordOK( );
3	if (!PwStatus)
4	puts("Access denied");
5	exit(–1);
6	else puts("Access granted");

Stack

Storage for Password (12 bytes) "123456789012"
Caller EBP—Frame Ptr main (4 bytes) "3456"
Return Addr Caller—main (4 bytes) "W▶*!" (return to line 6 was line 3)
Storage for PwStatus (4 bytes) '\0'
Caller EBP—Frame Ptr OS (4 bytes)
Return Addr of main—OS (4 bytes)

Figure 2.14. Program stack following buffer overflow using crafted input string

- Carefully crafted input string
  - j = 0x6A, ▶ = 0x10, * = 0x2A, ! = 0x21

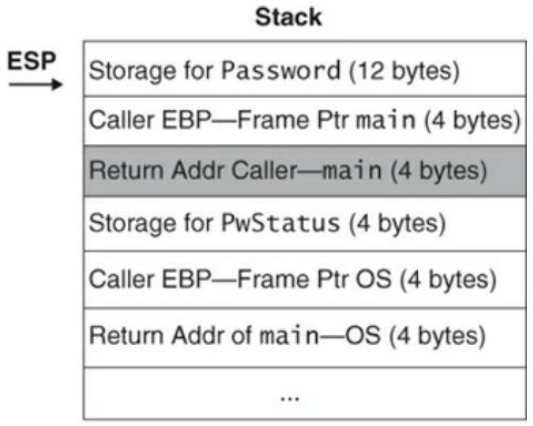
Code Injection

- when the return address is overwritten,
  - transferring control to a new address typically causes a trap and results in a corrupted stack
  - it is possible for an attacker to create a specially crafted string that contains a pointer to some malicious code
  - the malicious code runs with the permissions that the vulnerable program has
- malicious arguments
  - it must be accepted by the vulnerable program as legitimate input
  - the argument, along with other controllable inputs, must result in execution of the vulnerable code path
  - the argument must not cause the program to terminate abnormally before control is passed to the shellcode

Example

>> ./BufferOverflow < exploit.bin

```
bool IsPasswordOK(void) {
  char Password[12];

  gets(Password);
 r eturn 0 == strcmp(Password, "goodpass");
}

int main(void) {
  bool PwStatus;

  puts("Enter password:");
  PwStatus = IsPasswordOK();
  if (PwStatus == false) {
    puts("Access denied");
    exit(-1);
  }
}
```

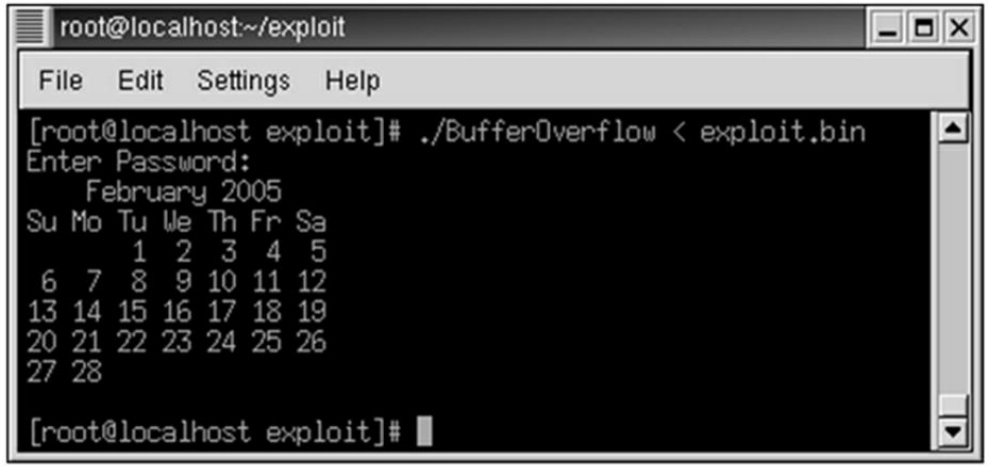


Note: The stack grows and shrinks as a result of function calls made by `IsPasswordOK(void)`.

```
/* buf[12] */
00 00 00 00
00 00 00 00
00 00 00 00

/* %ebp */
00 00 00 00

/* return address */
78 fd ff bf

/* "/usr/bin/cal" */
2f 75 73 72
2f 62 69 6e
2f 63 61 6c
00 00 00 00

/* null pointer */
74 fd ff bf

/* NULL */
00 00 00 00

/* exploit code */
b0 0b       /* mov  $0xb, %eax */
8d 1c 24    /* lea  (%esp), %ebx */
8d 4c 24 f0 /* lea  -0x10(%esp), %ecx */
8b 54 24 ec /* mov  -0x14(%esp), %edx */
cd 50       /* int  $0x50 */
```

Example

1. The first mov instruction is used to assign 0xB to the %eax register. 0xB is the number of the execve() system call in Linux
2. The three arguments for the execve() function call are set up in the subsequent three instructions. The data for these arguments is located on the stack.
3. The int $0x50 instruction is used to invoke execve(), which results in the execution of the Linux calendar program.




```
/* exploit code */
b0 0b        /* mov  $0xb, %eax */
8d 1c 24     /* lea  (%esp), %ebx */
8d 4c 24 f0  /* lea  -0x10(%esp), %ecx */
8b 54 24 ec  /* mov  -0x14(%esp), %edx */
cd 50        /* int  $0x50 */
```

Modified Example

fgets() instead of gets()

Bufferoverflow can occur here!

```
char buffer[128];

_Bool IsPasswordOK(void) {
  char Password[12];

  fgets(buffer, sizeof buffer, stdin);
  if (buffer[ strlen(buffer) - 1] == '\n')
    buffer[ strlen(buffer) - 1] = 0;
  strcpy(Password, buffer);
  return 0 == strcmp(Password, "goodpass");
}

int main(void) {
  _Bool PwStatus;

  puts("Enter password:");
  PwStatus = IsPasswordOK();
  if (!PwStatus) {
    puts("Access denied");
    exit(-1);
  }
  else
    puts("Access granted");
  return 0;
}
```

Modified Example

- Exploit Code

```
000: 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36  1234567890123456
010: 37 38 39 30 31 32 33 34 04 fc ff bf 78 78 78 78  78901234....xxxx
020: 31 c0 a3 23 fc ff bf b0 0b bb 27 fc ff bf b9 1f  1..#......'.....
030: fc ff bf 8b 15 23 fc ff bf cd 80 ff f9 ff bf 31  .....#.....'...1
040: 31 31 31 2f 75 73 72 2f 62 69 6e 2f 63 61 6c 0a  111/usr/bin/cal.
```

- Corrupted Stack

Row	Address	Content	Description
1	0xbffff9c0 -0xbffff9cf	"123456789012456"	Storage for Password (16 bytes) and padding
2	0xbffff9d0 -0xbffff9db	"789012345678"	Additional padding
3	0xbffff9dc	(0xbffff9e0)	New return address
4	0xbffff9e0	xor %eax,%eax	Sets eax to 0
5	0xbffff9e2	mov %eax,0xbffff9ff	Terminates pointer array with null pointer
6	0xbffff9e7	mov $0xb,%al	Sets the code for the execve() function call
7	0xbffff9e9	mov $0xbffffa03,%ebx	Sets ebx to point to the first argument to execve()
8	0xbffff9ee	mov $0xbffff9fb,%ecx	Sets ecx to point to the second argument to execve()
9	0xbffff9f3	mov 0xbffff9ff,%edx	Sets edx to point to the third argument to execve()
10	0xbffff9f9	int $80	Invokes execve() system call
11	0xbffff9fb	0xbffff9ff	Array of argument strings passed to the new program
12	0xbffff9ff	"1111"	Changed to 0x00000000 to terminate the pointer array and also used as the third argument
13	0xbffffa03 -0xbffffa0f	"/usr/bin/cal\0"	Command to execute

Arc Injection

- The *arc injection* technique involves transferring control to code that already exists in process memory
  - insert a new arc (control-flow transfer)
  - `system()` or `exec()` can be used to execute commands
  - can invoke multiple functions in sequence

Example

- An attacker may prefer arc injection over code injection
  - code already in memory on the target system
  - footprint can be significantly smaller
  - cannot be protected by memory-based protection schemes

```
#include <string.h>

int get_buff(char *user_input, size_t size){
  char buff[40];
  memcpy(buff, user_input, size);
  return 0;
}

int main(void) {
  /* ... */
  get_buff(tainted_char_array, tainted_size);
  /* ... */
}
```

Mitigation Strategies for Strings

- follow a defense-in-depth tactic that combines multiple strategies
  - a secure technique to string handling
  - + one or more runtime detection
- String handling
  - recommend selecting a single approach to handling character strings
  - string-handling functions can be categorized according to how they manage memory
    - Caller allocates, caller frees (C99, OpenBSD, C11 Annex K)
    - Callee allocates, caller frees (ISO/IEC TR 24731-2)
    - Callee allocates, callee frees (C++ `std::basic_string`)

C11 Annex K Bounds-Checking

- The first memory management model (caller allocates, caller frees)
- C11 Annex K provides alternative library functions
  - verify that output buffers are large enough for the intended result
  - return a failure indicator if they are not
  - all string results are null-terminated
- Ex) C11 Annex K defines
  - `strcpy_s(), strcat_s(), strncpy_s(), strncat_s(),…`
- published as ISO/IEC TR 24731-1 → later incorporated in C11 as a set of optional extensions
- need to define the `__STDC_LIB_EXT1__` macro
- C11 Annex K is a normative but optimal annex
  - need to check all your target platform
- if an invalid size is passed to one of the functions
  - it could still suffer from buffer overflow problems
- not implemented in gcc

Example

- Example: gets_s()

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
#include <stdlib.h>

void get_y_or_n(void) {
  char response[8];
  size_t len = sizeof(response);
  puts("Continue? [y] n: ");
  gets_s(response, len);
  if (response[0] == 'n')
    exit(0);
}
```

Runtime-constraint-handler function

- upon detecting an error, call a special *runtime-constraint-handler* function
  - might print an error message and/or abort the program
- the programmer can control which handler function is called
  - via the `set_constraint_handler_s()` function
  - can make the handler simply return if desired
- "ERR03-C. Use runtime-constraint handlers when calling functions defined by TR24731-1"
  - recommends installing a runtime-constraint handler to eliminate implementation-defined behavior

Example 2.6. gets_s() (Improved)

```
#define __STDC_WANT_LIB_EXT1__ 1
#include <stdio.h>
#include <stdlib.h>

void get_y_or_n(void) {
  char response[8];
  size_t len = sizeof(response);

  puts("Continue? [y] n: ");
  if ((gets_s(response, len) == NULL) || (response[0] == 'n')) {
     exit(0);
  }
}

int main(void) {
  constraint_handler_t oconstraint =
     set_constraint_handler_s(ignore_handler_s);
  get_y_or_n();
}
```

constraint handler

old constraint handler

Runtime-constraint-handler function

- If set to the `ignore_handler_s()` function,
  - any violation will return to its caller
- most bounds-checking functions return a nonzero `errno_t`.
  - but, the `gets_s()` function returns a null pointer (for compatibility)
- "ERR00-C. Adopt and implement a consistent and comprehensive error-handling policy"
- original constraint handler
  - in cases that the library function does set a constraint handler, the function must restore the original constraint handler before returning or exiting

Dynamic Allocation Functions

- the second memory management model (callee allocates, caller frees)
  - ISO/IEC TR 24731-2
- the use of such functions requires introducing additional calls to free the buffers later
- the functions described in ISO/IEC TR 24731-2 provide greater assurance
  - because buffers are always automatically sized to hold the data required
  - might suffer from denial-of-service attacks

Example

Example: getline()

```
#define __STDC_WANT_LIB_EXT2__ 1
#include <stdio.h>
#include <stdlib.h>

void get_y_or_n(void) {
  char *response = NULL;
  size_t len;

  puts("Continue? [y] n: ");
  if ((getline(&response, &len, stdin) < 0) ||
      (len && response[0] == 'n')) {
    free(response);
    exit(0);
  }
  free(response);
}
```

dynamically allocate the buffer

need to free the buffer

String stream

- fmemopen(), open_memstream()
- function prototypes

```
FILE *fmemopen(
  void * restrict buf, size_t size, const char * restrict mode
);
FILE *open_memstream(
  char ** restrict bufp, size_t * restrict sizep
);
```

String stream

- In the case of a stream opened by `open_memstream()`,
  - the memory area grows dynamically to accommodate write operations as necessary
  - it does require the caller to free the allocated memory (line 16)
- Dynamic allocation is often disallowed in safety-critical systems.
- Example

```
01  #include <stdio.h>
02
03  int main(void) {
04    char *buf;
05    size_t size;
06    FILE *stream;
07
08    stream = open_memstream(&buf, &size);
09    if (stream == NULL) { /* handle error */ };
10    fprintf(stream, "hello");
11    fflush(stream);
12    printf("buf = '%s', size = %zu\n", buf, size);
13    fprintf(stream, ", world");
14    fclose(stream);
15    printf("buf = '%s', size = %zu\n", buf, size);
16    free(buf);
17    return 0;
18  }
```

C++ std::basic_string

- C++ extraction operator `operator>>`
  - setting the field width eliminates the buffer overflow vulnerability
  - the issue of truncation
    - unexpected program behavior can occur
- the standard `std::string` class defined in ISO/IEC 14882
  - a specialization of the `std::basic_string` template on type `char`
- the basic_string class
  - uses a dynamic approach to strings
  - the "callee allocates, callee frees" memory management strategy → the most secure approach
  - Example

```
string str1 = "hello, ";
string str2 = "world";
string str3 = str1 + str2;
```

Example

- Example: std::cin

```
01  #include <iostream>
02  #include <string>
03  using namespace std;
04
05  int main(void) {
06    string str;
07
08    cin >> str;
09    cout << "str 1: " << str << '\n';
10  }
```

std:out_of_range

- the subscript member `std::string::operator[]` does not throw an exception

  string bs("01234567");

  size_t i = 10;

  bs[i] = '\0';

- the `at()` method is similar to `operator[]`, but throws an `out_of_range` exception if `pos >= size()`

- example

```
string bs("01234567");
try {
  size_t i = f();
  bs.at(i) = '\0';
}
catch (out_of_range& oor) {
  cerr << "Out of Range error: " << oor.what() << '\n';
}
```

c_str()

- to convert a `string` to a null-terminated byte string
  - the `c_str()` method
  - example

```
string str("abcd");
cout << strlen(str.c_str());
```

String-Handling Functions

- gets()
  - never invoke the `gets()` function
  - reads a line from standard input into a buffer until a terminating newline or end-of-file (EOF) is found
  - removed from C11
- alternative replacement function
  - C99
    - fgets() or getchar()

Example 2.9. fgets()

```
char buf[LINE_MAX];
int ch;
char *p;

if (fgets(buf, sizeof(buf), stdin)) {
  /* fgets succeeds, scan for newline character */
  p = strchr(buf, '\n');
  if (p) {
    *p = '\0';
  }
  else {
    /* newline not found, flush stdin to end of line */
    while (((ch = getchar()) != '\n')
          && !feof(stdin)
          && !ferror(stdin)
    );
  }
}
else {
  /* fgets failed, handle error */
}
```

newline char needs to be removed

fgets() and getchar()

- fgets()
  - The `fgets()` function retains the newline character
  - truncation: user input can be truncated
  - can result in a buffer overflow if the specified number of characters to input exceeds the length of the destination buffer
- getchar()
  - the `getchar()` function returns the next character from the input stream pointed to by `stdin`.
  - returns EOF
    - if the stream is at EOF
    - or if a read error occurs

getchar()

Example 2.10. getchar()

`int` type
(not `char` type)

```
char buf[BUFSIZ];
int ch;
int index = 0;
int chars_read = 0;

while (((ch = getchar()) != '\n')
        && !feof(stdin)
        && !ferror(stdin))
{
  if (index < BUFSIZ-1) {
    buf[index++] = (unsigned char)ch;
  }
  chars_read++;
} /* end while */
buf[index] = '\0';  /* null-terminate */
if (feof(stdin)) {
  /* handle EOF */
}
if (ferror(stdin)) {
  /* handle error */
}
if (chars_read > index) {
  /* handle truncation */
}
```

while (((ch = getchar()) != '\n') && ch != EOF)

C11 Annext K gets_s()

- the C11 `gets_s()` function
  - compatible to `gets()`
  - more secure version
  - reads from stdin
  - does not retain the newline character
- if an error occurs,
  - no input is performed
  - the char array is not modified
- otherwise
  - reads, at most, one less than the number of chars specified
  - a null char is written after the last char
- the gets_s() function succeeds only if it reads a complete line.
  - if a complete line cannot be read, the function returns NULL, sets the buffer to the null string, and clears input stream to the next newline char

Dynamic Allocation Functions

- the `getline()` function
  - described in ISO/IEC TR 24731-2
  - similar to `fgets()`, but has extra features
    - dynamically resize the destination buffer
    - returns the number of characters read
  - works only if the destination buffer is allocated with `malloc()`
  - the user must explicitly `free()` the buffer later

Example 2.12. getline()

```
int ch;
char *p;
size_t buffer_size = 10;
char *buffer = malloc(buffer_size);
ssize_t size;

if ((size = getline(&buffer, &buffer_size, stdin)) == -1) {
  /* handle error */
} else {
  p = strchr(buffer, '\n');
  if (p) {
    *p = '\0';
  } else {
    /* newline not found, flush stdin to end of line */
    while (((ch = getchar()) != '\n')
       && !feof(stdin)
       && !ferror(stdin)
       );
  }
}

/* ... work with buffer ... */

free(buffer);
```

Alternative functions for gets()

- Table 2.4

Table 2.4. Alternative Functions for `gets()`

	Standard/TR	Retains Newline Character	Dynamically Allocates Memory
`fgets()`	C99	Yes	No
`getline()`	TR 24731-2	Yes	Yes
`gets_s()`	C11	No	No

strcpy() and strcat()

- C99
  - the C standard `strncpy()` function is frequently recommended as an alternative
    - `strncpy()` is prone to null-termination errors and other problems
- OpenBSD
  - strlcpy() and strlcat()
    - the full size of the destination string as a size parameter
    - guarantee that the destination string is null-terminated
    - return the total length of the string they tried to create

C11 Annex K Bounds-Checking interfaces

- strcpy_s() and strcat_s()
  - return 0 on success
  - otherwise, a nonzero value is returned
  - enforces a variety of runtime constraints
    - if either s1 or s2 is a null pointer
    - if the size of destination buffer is zero, greater than RSIZE_MAX, or less than or equal to the length of the source string

String copy functions

- Table 2.5.

Table 2.5. String Copy Functions

	Standard/TR	Buffer Overflow Protection	Guarantees Null Termination	May Truncate String	Allocates Dynamic Memory
`strcpy()`	C99	No	No	No	No
`strncpy()`	C99	Yes	No	Yes	No
`strlcpy()`	OpenBSD	Yes	Yes	Yes	No
`strdup()`	TR 24731-2	Yes	Yes	No	Yes
`strcpy_s()`	C11	Yes	Yes	No	No

- Table 2.6.

Table 2.6. String Concatenation Functions

	Standard/TR	Buffer Overflow Protection	Guarantees Null Termination	May Truncate String	Allocates Dynamic Memory
`strcat()`	C99	No	No	No	No
`strncat()`	C99	Yes	No	Yes	No
`strlcat()`	OpenBSD	Yes	Yes	Yes	No
`strcat_s()`	C11	Yes	Yes	No	No

strncpy() and strncat()

- has an additional `size_t` parameter `n`
- `strncpy()` is not guaranteed to null-terminate the destination string.
  - prone to string termination errors
- `strncat(s1, s2, n)`
  - the min size of the `s1` buffer should be `strlen(s1) + n + 1`
  - the last argument to `strncat()` should not be the total buffer length
  - `strncat(dest, source, dest_size - strlen(dest) - 1)`
- there is no status code when the resulting string is truncated

strncpy_s() and strncat_s()

- strncpy_s()

```
errno_t strncpy_s(
  char * restrict s1,
  rsize_t s1max,
  const char * restrict s2,
  rsize_t n
);
```

- if a runtime-constraint violation occurs, the destination array is set to the empty string
- null terminator is added
- returns 0 to indicate success
- if the input arguments are invalid, it returns a nonzero value
- no truncation!!!

strncpy_s()

Example

```
char src1[100] = "hello";
char src2[7] = {'g','o','o','d','b','y','e'};
char dst1[6], dst2[5], dst3[5];
errno_t r1, r2, r3;
```

- r1 = strncpy_s(dst1, sizeof(dst1), src1, sizeof(src1));
  - r1 = 0, dst1 = “hello\0”
- r2 = strncpy_s(dst2, sizeof(dst2), src2, 4);
  - r2 = 0, dst2 = “good\0”
- r3 = strncpy_s(dst3, sizeof(dst3), src1, sizeof(src1));
  - r3 = nonzero value, dst3[0] = ‘\0’

Summary of strcpy() and strcat()

- Summary tables

Table 2.7. Truncating Copy Functions

	Standard/TR	Buffer Overflow Protection	Guarantees Null Termination	May Truncate String	Allocates Dynamic Memory	Checks Runtime Constraints
`strncpy()`	C99	Yes	No	Yes	No	No
`strlcpy()`	OpenBSD	Yes	Yes	Yes	No	No
`strndup()`	TR 24731-2	Yes	Yes	Yes	Yes	No
`strncpy_s()`	C11	Yes	Yes	No	No	Yes

	Standard/TR	Buffer Overflow Protection	Guarantees Null Termination	May Truncate String	Allocates Dynamic Memory	Checks Runtime Constraints
`strncat()`	C99	Yes	No	Yes	No	No
`strlcat()`	OpenBSD	Yes	Yes	Yes	No	No
`strncat_s()`	C11	Yes	Yes	No	No	Yes

Runtime Protection Strategies

- there is a danger that programmer can believe they have solved the problem by using an incomplete detection and recovery strategy
- Buffer overflow mitigation strategies can be classified according to which component of the system provides the mitigation mechanism
  - the developer via input validation
  - the compiler and its associated runtime system
  - the operating system

Input validation

- best way to mitigate buffer overflows

```
void f(const char *arg) {
 char buff[100];
 if (strlen(arg) >= sizeof(buff)) {
   abort();
 }
 strcpy(buff, arg);
 /* ... */
}
```

- any data that arrives at a program interface across a trust boundary requires validation
  - ex) argv and argc, environment variables, data read from sockets/files/devices …

Object Size Checking

- GCC provides limited functionality to access the size of an object given a pointer into that object
  - GCC 4.1: `size_t __builtin_object_size(void *ptr, int type)`
    - ptr: pointer into any object (requires to point to the start of the object)
    - type: 0~3
      - 0 or 2: the referenced object is the largest object containing the pointed-to byte
      - otherwise: the object is the smallest object
    - returns the number of bytes
    - example

```
struct V { char buf1[10]; int b; char buf2[10]; } var;
void *ptr = &var.b;
```

      - __builtin_object_size(ptr, 0)
      - __builtin_object_size(ptr, 1)

Object Size Checking

- __builtin_object_size()

Table 2.9. Behavior Effects of `type` on `__builtin_object_size()`

Value of type Argument	Operates on	If Unknown, Returns
0	Maximum object	(size_t) -1
1	Minimum object	(size_t) -1
2	Maximum object	(size_t) 0
3	Minimum object	(size_t) 0

- `__builtin_object_size()` function is used when `_FORTIFY_SOURCE` is defined

memcpy()	strcpy()	strcat()	sprintf()	vsprintf()
memmove()	strncpy()	strncat()	snprintf()	vsnprintf()
memset()	fprintf()	vfprintf()	printf()	vprintf()

Object Size Checking

- example

Cannot determine the size

src_end = the number of bytes including '\0'

```
char dest[BUFFER_SIZE];
char *src = /* valid pointer */;
size_t src_end = __builtin_object_size(src, 0);
if (src_end == (size_t) -1 && /* don't know if src is too big */
    strlen(src) < BUFFER_SIZE) {
  strcpy(dest, src);
} else if (src_end <= BUFFER_SIZE) {
  strcpy(dest, src);
} else {
  /* src would overflow dest */
}
```

Visual Studio Compiler-Generated Runtime Checks

- the /RTCs compiler flag turns on runtime checks for the following errors:
  - overflows of local variables such as arrays
  - use of uninitialized variables
  - stack pointer corruption
- these flags can be tweaked on or off
  - #pragma runtime_checks(“s”, off)
  - #pragma runtime_checks(“s”, restore)
- Visual C++ 2010: /GS
  - #pragma strict_gs_check(on)

Stack-Smashing Protector (ProPolice)

- GCC 4.1 introduced the Stack-Smashing Protector (SSP) feature.
  - reorders local variables to place buffers after pointers
  - copies pointers in function arguments to an area preceding local variable buffers
- GCC command-line arguments
  - `-fstack-protector, -fstack-protector-all`
  - `-fno-stack-protector, -fno-stack-protector-all`
- SSP works by introducing a canary to detect changes

Stack-Smashing Protector (ProPolice)

SSP stack structure

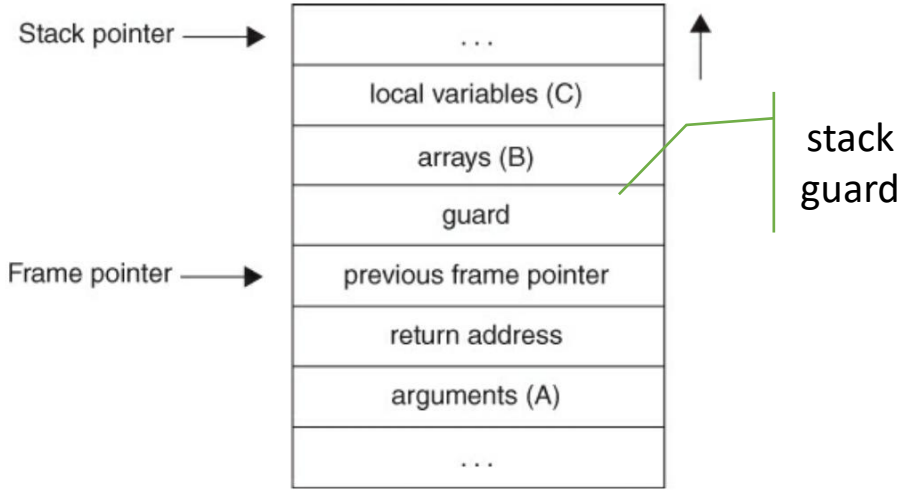


Figure 2.18. Stack-Smashing Protector (SSP) stack structure

- location (A) has no array or pointer variables
- location (B) has arrays or structures that contain arrays
  - place the guard after location (B)
- location (C) has no arrays

Operating System Strategies

Address space layout randomization (ASLR)

- to prevent arbitrary code execution
- Linux: first introduced in 2000, Ubuntu (since 2008), Debian (since 2007)
  - sysctl –w kernel.randomize_va_space=2
  - execute this command during the boot process
    - 0: Turns off ASLR completely
    - 1: Turns on ASLR for stacks, libraries, and position-independent binary programs
    - 2: Turns on ASLR for the heap, too
- available on Windows since Vista
  - developers of custom DLLs and EXEs must opt in to support ASLR using the `/DYNAMICBASE` linker option

Operating System Strategies

- Nonexecutable stacks
  - do not allow an attacker to run executable code on the stack
- W ^ X (W xor X)
  - OpenBSD, Windows, Linux, OS X
  - no part of the process address space is both writable and executable
  - supported by the use of a No eXecute (NX) bit on several CPUs
    - the NX bit enables memory pages to be marked as *data*
    - NX on AMD, XD (eXecute Disable) on Intel CPUs, XN (eXecute Never) on ARM version 6 and later
- Data Execution Prevention (DEP)
  - implementation of the W ^ X policy for Microsoft Visual Studio
  - uses NX technology to prevent the execution of instructions stored in data segments
  - no performance impact on Windows
  - to enable DEP, you should link your code with `/NXCOMPACT` or call `SetProcessDEPPolicy()`

Operating System Strategies

- PaX
  - Linux kernel security patch
  - nonexecutable stack
  - ASLR
  - can use NX technology
  - can cause a denial of service (DoS)
  - now part of the `grsecurity` project
- StackGap
  - introducing a randomly sized gap of space upon allocation of stack memory

- A possible Safe-Secure C/C++ (SSCC) implementation

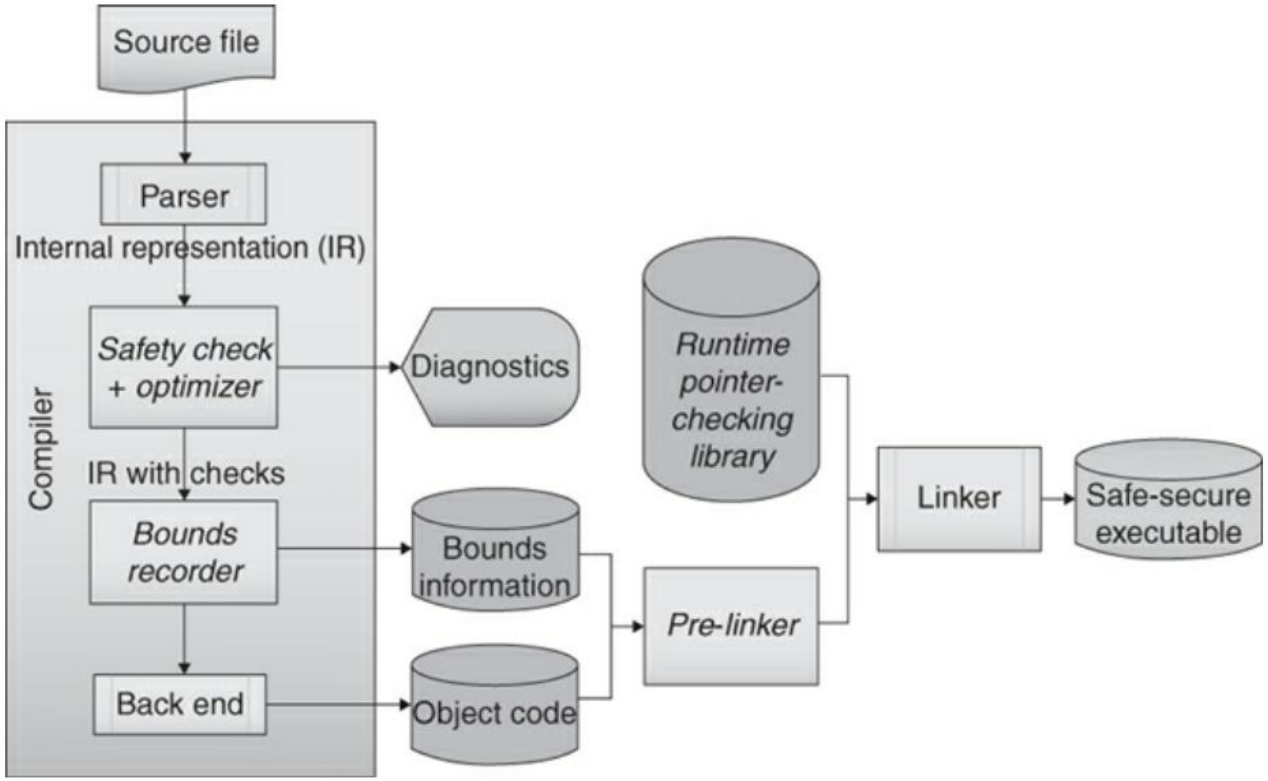


Figure 2.19. A possible Safe-Secure C/C++ (SSCC) implementation

Notable Vulnerabilities

- Remote Login (rlogin)
  - buffer overflow: copying the TERM environment variable into an array of 1,024 characters declared as a local stack variable
- Kerberos
  - a network authentication protocol
  - http://web.mit.edu/Kerberos/
  - buffer overflow in the `krb_rd_req()` function
    - an attacker can gain root access over the network

Summary

- buffer overflows occur frequently in C and C++ because
  1. define strings as null-terminated arrays of charcters
  2. do not perform implicit bounds checking
  3. provide standard library calls for strings that do not enforce bounds checking
- a common mitigation strategy is to adopt a new library
- practical mitigation strategies include static analysis and dynamic analysis
- runtime solutions
  - bounds checkers, canaries, safe libraries
- Defense-in-depth strategy !

Q&A